

# Laboratorio 6

Ignacio Méndez Álvarez (22613) y Diego Soto Flores (22737)

## Task 1 — Arquitecturas modernas aplicadas al diagnóstico de enfermedades en hojas de mango

### 1. Por qué una red secuencial extremadamente profunda tipo VGG (150 capas) no sería una buena decisión para este problema y cómo ResNet resuelve la situación

La idea de que más profundo siempre es mejor suena lógica, pero ya en la práctica una red secuencial tipo VGG con unas 150 capas probablemente no entrenaría bien. Esto pasa por dos problemas conocidos, el desvanecimiento del gradiente y el fenómeno de degradación.

El desvanecimiento del gradiente aparece durante backpropagation. Si vemos la red como una composición de funciones:

$$x_L = f_L(f_{L-1}(\dots f_1(x)))$$

entonces el gradiente que llega a las primeras capas depende de multiplicar muchas derivadas:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{i=2}^L \frac{\partial x_i}{\partial x_{i-1}}$$

Cuando esas derivadas son menores que 1, el producto se vuelve muy pequeño mientras más capas tenga la red. En una red de 150 capas, las primeras capas casi no reciben señal de aprendizaje, y eso es malo porque son las que detectan bordes, texturas o pequeñas manchas en las hojas.

Además aparece el fenómeno de degradación, o sea que al agregar más capas, el error de entrenamiento puede llegar a empeorar en lugar de mejorar.

ResNet soluciona esto usando conexiones residuales. En vez de aprender directamente una función  $H(x)$ , el bloque aprende una función residual:

$$F(x) = H(x) - x$$

por lo que la salida del bloque se vuelve:

$$H(x) = F(x) + x$$

Esto hace que la red solo tenga que aprender una pequeña corrección sobre la entrada, lo

que facilita el entrenamiento.

Además, las conexiones residuales ayudan al flujo del gradiente:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H} \left( \frac{\partial F}{\partial x} + 1 \right)$$

Ese término +1 crea un camino directo para el gradiente, evitando que desaparezca y permitiendo entrenar redes mucho más profundas sin que el entrenamiento colapse.

## 2. Por qué la arquitectura Inception es especialmente adecuada para analizar enfermedades en hojas de mango

Las enfermedades en hojas de mango tienen un problema interesante, porque no siempre se ven igual ni al mismo tamaño. Algunas aparecen como puntitos pequeños, mientras que otras cubren zonas grandes de la hoja. Entonces el modelo necesita detectar patrones a distintas escalas.

En redes convolucionales, el tamaño del filtro define qué tan grande es el patrón que se puede detectar. Por ejemplo, filtros pequeños como  $3 \times 3$  capturan detalles finos, mientras que filtros más grandes como  $5 \times 5$  capturan estructuras más amplias. Si usamos solo un tamaño de filtro en toda la red, el modelo termina analizando la imagen solo a una escala, lo que limita lo que puede aprender.

La arquitectura Inception soluciona esto aplicando varios filtros en paralelo dentro del mismo bloque. Si  $x$  es la entrada del módulo, la salida puede escribirse como:

$$y = \text{concat} (f_{1 \times 1}(x), f_{3 \times 3}(x), f_{5 \times 5}(x), f_{\text{pool}}(x))$$

Cada rama analiza la imagen con un tamaño de filtro distinto. En el caso de las hojas de mango, esto permite detectar manchas pequeñas, patrones medianos o zonas grandes de infección al mismo tiempo, lo cual es justo lo que necesitamos para este problema biológico.

Ahora, el problema es que tener varias convoluciones en paralelo puede volver el modelo muy caro computacionalmente. El costo aproximado de una convolución se puede expresar como:

$$K^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

donde  $K$  es el tamaño del kernel,  $C_{in}$  el número de canales de entrada y  $C_{out}$  el número de filtros.

Si aplicamos filtros grandes directamente sobre tensores con muchos canales, el costo crece bastante. Por eso Inception usa convoluciones  $1 \times 1$  antes de los filtros grandes. Estas funcionan como una reducción de dimensionalidad, bajando el número de canales antes de hacer las operaciones más caras.

En la práctica, esto significa que podemos tener un modelo que detecte patrones complejos

en las hojas sin disparar el costo de cómputo. Si pensamos en infraestructura real esto también ayuda a mantener bajo control el presupuesto de la startup, que al final también es parte importante de la decisión arquitectónica.

### 3. Cómo funciona MobileNet y por qué su diseño es adecuado para ejecutar el modelo en teléfonos utilizados por agricultores

En el proyecto AgriTech el modelo se va a ejecutar directamente en teléfonos Android, no en servidores potentes. Eso significa que hay limitaciones fuertes de memoria, batería y poder de cómputo. Arquitecturas como ResNet o Inception pueden ser muy buenas, pero también son relativamente pesadas para este tipo de dispositivos.

MobileNet se diseñó justamente para este escenario. Su idea principal es reemplazar la convolución estándar por una operación más eficiente llamada depthwise separable convolution, que separa el filtrado espacial de la combinación entre canales.

En una convolución normal, cada filtro opera sobre todos los canales al mismo tiempo. El costo aproximado es:

$$D_k^2 \cdot M \cdot N \cdot H \cdot W$$

donde  $D_k$  es el tamaño del kernel, M el número de canales de entrada y N el número de filtros.

MobileNet divide esta operación en dos pasos. Primero aplica una depthwise convolution, que filtra cada canal por separado:

$$D_k^2 \cdot M \cdot H \cdot W$$

Luego aplica una pointwise convolution (una convolución  $1 \times 1$ ) que combina la información entre canales:

$$M \cdot N \cdot H \cdot W$$

Entonces el costo total queda:

$$D_k^2 \cdot M \cdot H \cdot W + M \cdot N \cdot H \cdot W$$

Esto reduce bastante el costo computacional comparado con una convolución estándar, lo que hace que el modelo sea mucho más ligero.

El precio que se paga es que el modelo pierde algo de capacidad expresiva, porque las relaciones espaciales y entre canales ya no se aprenden al mismo tiempo. Aun así, para esta aplicación el trade-off vale la pena porque es mejor tener un modelo un poco menos complejo pero que funcione rápido en el teléfono del agricultor, incluso sin internet.